SAISREE DENDI

Dallas, TX | (214) 616-3779 | saisreedendi@gmail.com | LinkedIn

PROFILE SUMMARY

Full-stack Software Developer with 6+ years of experience building Java backend services, React frontends, and cloud applications for hardware, healthcare, and financial-services teams. Works in Java and SQL every day, builds REST and GraphQL APIs, and uses Jenkins and GitHub Actions to test and release code with confidence. Has built clear, easy-to-use web screens with React and TypeScript, reliable data pipelines with Apache Spark, and secure, HIPAA-compliant systems that run on AWS and GCP. Works well in Agile teams and enjoys partnering with business, QA, and infrastructure teams to turn requirements into working software.

TECHNICAL SKILLS

Programming Languages	Java, SQL, JavaScript, TypeScript, Shell Scripting (Bash)
Core CS Fundamentals	Data Structures & Algorithms, Object-Oriented Programming (OOP)
Frontend Development	React, TypeScript, JavaScript, HTML5, CSS3, Responsive Web Design
Backend & APIs	Node.js, RESTful & GraphQL APIs, Microservices, FHIR APIs, HL7 Standards
Databases	SQL, MongoDB, BigQuery
ETL & Data Pipelines	ETL Pipeline Design, Apache Spark, Kafka, Unix/Linux Shell Scripting, NDM File Transfer, AutoSys Job Scheduling, Enterprise Data Warehouse (EDW)
Cloud Platforms	AWS (EC2, S3, Lambda, IAM), GCP (Dataflow, BigQuery, Cloud Storage)
CI/CD & DevOps	Jenkins, GitHub Actions, CI/CD Pipeline Design, Automated Build/Test/Deploy Workflows
Testing & Quality Assurance	Unit Testing, Integration Testing, Functional Testing, Regression Testing, API Testing
Security & Compliance	HIPAA Compliance, TLS/AES Encryption, Role-Based Access Control (RBAC), Audit Logging, AML/KYC Compliance
Hardware & Systems Validation	Chipset Driver Testing, BIOS/Firmware Validation, PCIe, GPU Telemetry, Multi-Display Systems
Development Methodologies	Agile/Scrum, Requirement Gathering & Analysis, Technical Documentation, Code Reviews
Tools & Version Control	Git, GitHub, Unix/Linux, Debugging

PROFESSIONAL EXPERIENCE

Java Developer, AMD, Austin, TX **Feb 2025 – Present**

Project description: Java developer on the hardware validation team, building the Java applications and services engineers use to test chip and driver releases before they go out. The work includes Java test automation for chipset drivers, BIOS/firmware, and multi-display setups; Java REST services that collect and serve hardware test data; React screens that show that data; Jenkins and GitHub Actions pipelines that build and test every Java commit; and AWS servers that run the tests.

Frontend

- Built internal dashboards with React, TypeScript, HTML5, and CSS3 that show live GPU hardware data coming from the Java services, so engineers can check results quickly without opening raw log files.
- Built a lab management screen in React and TypeScript on top of Java REST APIs, so the team can set up test computers, install operating systems, and flash firmware without working on each machine by hand.
- Added charts with React and JavaScript that show graphics performance and frame-time data returned by the Java services, which the team uses in launch and cross-team review meetings.
- Connected the React screens to the Java backend through REST APIs, so test results and hardware data on screen stay current as new runs finish.
- Added search and filter options in TypeScript so engineers can find a test run by hardware setup, firmware version, or test type in a few clicks.
- Met with the hardware validation engineers to learn what data they check first, then shaped the Java API responses and the screens so failed runs are easy to spot.

Backend

- Write and maintain the Java test frameworks that check chipset drivers, BIOS/firmware, and overall system behavior for each new hardware and driver release.
- Built Java services with REST APIs that collect hardware data such as PCIe bandwidth, GPU memory use, and thermal throttling, so the driver team can trace a problem back to its cause faster.
- Wrote Java test cases with object-oriented design, using inheritance and reusable classes, for multiplemonitor setups, power state changes, and heavy graphics workloads.
- Used Java collections, multithreading, and exception handling to run many tests at the same time and keep one failed test from stopping the whole run.
- Added Java logging and nightly reports so the team can see which tests pass and fail over time and spot tests that fail at random.
- Created a shared Java library of test helpers, kept in Git, along with Bash scripts that set up each test machine, so teammates can reuse common test code instead of writing it again.
- Debugged Java test failures by reading logs and stack traces, then fixed the cause so the same failure does not come back in later releases.

Cloud

- Run the Java build agents and test machines on AWS EC2, managed through Jenkins and GitHub Actions, so every driver and firmware commit is built and tested automatically.
- Set up Jenkins and GitHub Actions pipelines that compile the Java code, run the unit, functional, regression, and sanity tests, and report the result on every new commit.
- Store test results, hardware logs, and past test history in AWS S3 so the team can look back and see how results change over time.

- Use AWS IAM to limit access to the cloud testing tools and dashboards to the validation team, which keeps test data and settings protected.
- Watch AWS usage and cost for the test environment and lower the computing resources when testing slows down, to avoid paying for what is not needed.

Environment: Java, Java REST services, React, TypeScript, JavaScript, HTML5, CSS3, Jenkins, GitHub Actions, Git, Bash, AWS (EC2, S3, IAM), PCIe, GPU telemetry, BIOS/firmware validation, multi-display test systems.

Software Developer, Citi, Irving, TX. **Nov 2023 – Aug 2024**

Anti-Money Laundering (AML TM)

Project description: Worked on the Anti-Money Laundering Transaction Monitoring program, which pulls customer, KYC, and daily transaction data from the bank's Enterprise Data Warehouse, checks it, and flags suspicious activity for the compliance team to review. The work includes Java and Apache Spark data pipelines, automated job scheduling and file transfers, production support, and moving some pipeline parts to the cloud.

Frontend

- Built daily reports with Java and SQL that summarize transaction and risk activity, so the Product and Operations teams can see what happened without pulling the data themselves.
- Set up the Java delivery pipelines to produce clean, consistent audit trails and risk summaries that the compliance team can review and keep on record.
- Worked with analysts to decide which fields each report needs and wrote the SQL queries to match, so the reports answer the questions reviewers actually ask.
- Created reusable Java report templates, which cut setup time when a new type of AML review needed its own report.
- Scheduled report delivery in AutoSys so stakeholders receive current numbers on time every day.

Backend

- Built and maintained ETL pipelines with Java and Apache Spark that pull, clean, and check large amounts of customer (KYC), authorization, and transaction data from the Enterprise Data Warehouse, and used Kafka to stream transaction events into the same pipelines.
- Wrote Unix Shell scripts and set up NDM file transfers to automate the daily data intake, which removed a manual step that used to slow down the start of each day's transaction review.
- Set up and managed AutoSys jobs that run the daily processing, data checks, and risk-checking scripts in the right order and on time.
- Wrote reusable Java validation code that compares records from different data sources to help find possible fraud and give compliance a clear audit trail.
- Worked with compliance analysts to adjust the detection rules and limits in the Java pipeline code, which made the flagged transactions more accurate.
- Kept production scripts and Java jobs running on RHEL Linux by fixing issues, checking data quality, and making small improvements as needs changed.
- Covered on-call support for important AML jobs, fixing AutoSys and Apache Spark failures and data problems within the team's response times.

Cloud

- Helped move selected pipeline parts from on-premise servers to AWS, using EC2 for computing and S3 for storage, and worked with the infrastructure team on planning and testing.
- Used AWS S3 to store processed transaction data and pipeline logs so the compliance and audit teams could get them when needed.

- Tried GCP Dataflow and BigQuery on sample transaction data to see if they could replace some of the on-premise AutoSys jobs, using Dataflow to process the data and BigQuery to query it.
- Set up basic AWS monitoring on the cloud parts of the pipeline so failures were noticed early instead of at the next scheduled run.
- Worked with the infrastructure and security teams to set up AWS IAM access that met the bank's data security and compliance rules.

Environment: Java, SQL, Apache Spark, Unix Shell Scripting (Bash), Kafka, NDM File Transfer, AutoSys, RHEL/Enterprise Linux, Enterprise Data Warehouse (EDW), AWS (EC2, S3, IAM), GCP (Dataflow, BigQuery), Agile/Scrum.

Software Developer, Luma Health, Dallas, TX **Jan 2022 – Oct 2023**

Project description: Worked on Luma Health's patient-engagement platform from start to finish, covering the frontend, backend, and cloud. The work includes React screens for patients and clinic staff; Java and Node.js services that stay in sync with medical record systems through FHIR and HL7; HIPAA-compliant security across the system; and AWS hosting that keeps the platform available.

Frontend

- Built responsive, accessible screens with React, TypeScript, HTML5, and CSS3 where patients book appointments and fill out intake forms on a phone or a computer.
- Built admin dashboards with React, TypeScript, and REST APIs so clinic staff can manage appointments, see message status, and review how the clinic is doing.
- Used React with REST and GraphQL APIs to build charts that show patient response rates, missed appointments, and busy spots in the schedule.
- Wrote reusable React components in TypeScript that the team shared across features, which cut repeated code and made new patient screens faster to build.
- Ran feedback sessions with clinic staff and used what they said to improve the React dashboard screens and how easy they are to use.

Backend

- Built two-way data pipelines with Java and Node.js that keep patient records, appointments, and clinical details in sync between the platform and medical record systems using FHIR APIs and HL7.
- Built Java microservices with REST APIs, SQL, and MongoDB for smart scheduling, text and email reminders, and digital intake forms that handle a large number of requests every day.
- Added TLS/AES encryption, role-based access control (RBAC), and audit logging in Java to keep patient data private and HIPAA-compliant.
- Wrote automated unit, integration, and API tests that run in Jenkins and GitHub Actions on every release, so updates do not break services that patients depend on.
- Worked with medical record vendor teams to check FHIR and HL7 field mappings in the Java and Node.js services and to fix data that did not match between systems.
- Used Kafka to pass appointment and reminder events between the Java services, so messages are sent on time even when many arrive at once.
- Used Java and Node.js debugging and service logs to trace problems reported by clinic staff, then fixed them quickly to keep clinics running.

Cloud

- Deployed and ran the Java backend services on AWS using EC2, S3, and Lambda, so patient-facing apps stay available when clinics need them.
- Used AWS IAM, TLS/AES encryption, and audit logging to keep patient data protected and HIPAAcompliant in every environment.

- Set up Jenkins and GitHub Actions to deploy the Java services to AWS automatically with each release, so every release goes out the same way.
- Set up AWS staging environments so the team could test changes safely before they reached the live platform.
- Watched AWS usage and cost and adjusted the settings so the platform ran well without paying for extra resources.

Environment: Java, Node.js, React, TypeScript, HTML5, CSS3, REST/GraphQL APIs, FHIR, HL7, SQL, MongoDB, AWS (EC2, S3, Lambda, IAM), TLS/AES encryption, RBAC, Kafka, Jenkins, GitHub Actions.

Software Developer, Quadrant Resource Pvt Ltd, Madhapur, Hyderabad, India Oct 2019 – Dec 2021

Project description: Built and released business applications and ETL pipelines in Java for client projects. The work includes REST APIs and microservices that connect older systems to newer cloud tools, the SQL behind the data layer, pipelines that move data between several databases, and support for moving on-premise data jobs to the cloud.

Frontend

- Built the SQL and Java reporting layer behind the daily reports that clients rely on to run their operations.
- Built Java REST APIs that send data to client-facing applications in the format each client asked for.
- Wrote SQL queries that feed the analytical reports clients use in their daily work.
- Worked with business analysts to decide what each report and API response should show, so it matched how clients really use the data.
- Added SQL and Java checks that compare report data with the source systems before delivery, so clients receive correct numbers.

Backend

- Built, tested, and released business applications in Java and SQL based on each client's requirements.
- Used Java, REST APIs, and microservices to connect older business systems with newer cloud services so data could move between them.
- Turned business requirements into clean, easy-to-maintain Java code using object-oriented design and sound data structures and algorithms to keep the code fast.
- Took part in code reviews and pair programming through Git, and used Bash scripts to automate repeated build and deploy steps, to keep Java coding standards the same across the team.
- Wrote Java unit tests for the applications so bugs were caught early, before the client releases.
- Wrote and tuned SQL queries, stored procedures, joins, and indexes so the system could handle large amounts of data faster.
- Built Java ETL pipelines that extract, change, and load data across several databases, and wrote technical documentation of how the data is mapped so audits and later changes are easier.
- Tested the Java pipelines and SQL data flows from start to finish with the QA and business teams before each release.

Cloud

- Deployed and supported Java services and ETL jobs on AWS EC2 and S3 so enterprise data jobs run on schedule.
- Used AWS S3, GCP Cloud Storage, Dataflow, and BigQuery, along with cloud-hosted SQL databases, to bring data from several sources together for client projects.
- Worked with infrastructure teams to move on-premise Java and SQL data jobs to AWS and GCP, which made them more reliable and easier to scale.

- Set up basic monitoring on cloud jobs so failures were noticed quickly instead of waiting for the next run.
- Helped infrastructure teams test cloud settings before rollout so new setups worked as expected on golive day.

Environment: Java, SQL, RESTful APIs, Microservices, ETL Pipelines, Git, AWS (EC2, S3), GCP (Cloud Storage, Dataflow, BigQuery), Bash, CI/CD, Agile/Scrum.

EDUCATION

Master of Science in Data Engineering
Southern Methodist University - Dallas, TX

Bachelor of Engineering in Electrical and Electronics Engineering
Stanley College of Engineering and Technology for Women - Hyderabad, India

PUBLICATION

Dual-Axis Tracking System with Load Integration for Enhanced Power Generation - jespublication.com/issue.php?cid=26scid=108